CASE STUDY 14

Race Condition in xv6 Shared Data

Slide-by-slide speaking notes for the rebuilt presentation

Operating Systems | Department of CSE

Chaitanya Bharathi Institute of Technology

Prepared by

Student	Roll number
L Nithish Kumar Goud	160124733327
G Ishan	160124733322
Santhosh	160124733332
Venkat	160124733340

27 October 2026

Presenter guide

This script follows the nine-slide deck and keeps the focus on one shared structure and one fix: xv6's physical-page free list and a spinlock around the complete pop operation. The C program is a user-space model, not a modified xv6 kernel.

Slides	Presenter	Focus
1-2	Venkat	Case question, goal, selected structure
3-4	G Ishan	Free-list invariant and race schedule
5, 7	L Nithish Kumar Goud	Runnable C model and recorded output
6, 8	Santhosh	Spinlock operation and comparison
9	Venkat	Conclusion and scope limits

Table 1. Speaking assignments for the nine-slide deck.

Accuracy notes

- The two-thread barrier forces a specific interleaving in the host demonstration.
- Atomic pointer loads/stores avoid a C data race but do not make the whole free-list pop exclusive.
- The spinlock run is the corrected case. The program does not boot xv6 or modify kernel/kalloc.c.
- The output is from the recorded run in evidence/run_output.txt.

Slide 1 | Case Study 14: Race Condition in xv6 Shared Data

Presenter: Venkat

Spoken script: Good morning. We are presenting Case Study 14 on race conditions in xv6 shared data. Our focus is the physical-page free list. We will show how two concurrent allocations can act on the same old head, then run a small C model that reproduces that outcome and applies a spinlock fix.

Slide 2 | Assignment and selected structure

Presenter: Venkat

Spoken script: The assignment asks us to identify shared kernel data, study its critical section and lock, and compare behavior with and without synchronization. We chose the physical-memory free list, kmem.freelist. Our check is simple: a page must appear exactly once either in the free list or as an allocation.

Slide 3 | The xv6 free-list invariant

Presenter: G Ishan

Spoken script: The free list is a linked list of available physical pages. kalloc removes its head and kfree adds a page back at the head. In xv6, both operations use kmem.lock around the pointer changes. That scope matters because the read and update depend on one another.

Slide 4 | One harmful interleaving

Presenter: G Ishan

Spoken script: The table shows a possible schedule. CPU 0 reads P0, then CPU 1 also reads P0 before either advances the head. Both then store P1 as the new head and return P0. The same physical page now has two owners. This is the schedule we force in our host program, not a trace captured from xv6 hardware.

Slide 5 | C code that reproduces the race

Presenter: L Nithish Kumar Goud

Spoken script: In the unlocked worker, each thread loads the free-list head and saves its next pointer. The barrier waits until both threads have read the old head. Then both publish P1 and both return P0. We use atomic pointer operations so this demonstration does not depend on undefined C data races. The atomic accesses still do not protect the full operation.

Slide 6 | Spinlock fix

Presenter: Santhosh

Spoken script: The corrected worker acquires one spinlock before reading the head. It advances the head while still holding that lock, then releases it. The second worker enters afterward and sees P1. This preserves the page ownership invariant because the lock covers the whole read-and-update sequence.

Slide 7 | Recorded run

Presenter: L Nithish Kumar Goud

Spoken script: We built and ran the program with make run under WSL Ubuntu 24.04 using GCC 13.3.0. The unlocked case read P0 twice and returned P0 twice. The invariant audit found only two of three pages present exactly once. The spinlock case returned P0 and P1 and accounted for all three pages exactly once.

Slide 8 | Comparison and interpretation

Presenter: Santhosh

Spoken script: The two cases differ in whether the complete free-list pop has exclusive access. The unlocked run intentionally reproduces one harmful interleaving. The spinlock run preserves the invariant. This is a controlled demonstration, so it shows that the race can happen and how the lock prevents this schedule; it does not measure how frequently arbitrary races occur.

Slide 9 | Conclusion and scope

Presenter: Venkat

Spoken script: The main lesson is that mutual exclusion must cover every dependent read and write that preserves shared state. xv6 uses kmem.lock for its physical-page allocator. Our runnable C program mirrors that logic and makes the before-and-after behavior visible. It is a user-space teaching model, not a booted xv6 experiment. Thank you.

Source note: local xv6-riscv kernel/kalloc.c and kernel/spinlock.c at revision ae7cff64d1cd1153e98794d19c46235ead934cf2; recorded host run in evidence/run_output.txt.